ServiceNow ↔ Microsoft Intune Integration — Device Inventory & CMDB Sync

Implementation Solution Guide

Version 1.0.0 · 2026-07-30

Contents

Executive Summary — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync) 4

The situation . . . 4

What this delivers . . . 4

Why it matters . . . 4

What it is not (scope discipline) . . . 4

Approach . . . 4

1. Solution Overview — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync) 5

What this integration is . . . 5

How it works (the data flow) . . . 5

The supported product path . . . 6

Replacing the Workspace ONE (AirWatch) feed . . . 6

What data flows in . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 7

Out of scope (deferred to dedicated solutions — referenced, not reproduced) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync) 7

Part A — Microsoft (Intune / Entra / Graph) . . . 8

Part B — ServiceNow . . . 9

Government / sovereign cloud (GCC High / DoD) — read before configuring a Gov tenant . . . 10

Checklist . . . 11

Global Requirements — Intune Solutions (category-level) 11

G1. Microsoft Entra ID tenant . . . 11

G2. Microsoft Intune subscription . . . 11

G3. Administrative roles . . . 11

G4. Device enrollment in Intune . . . 12

G5. Pilot security group . . . 12

G6. Supported Windows baseline . . . 12

G7. Network access to Microsoft cloud endpoints . . . 12

3. Implementation Guide — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync) 12

PART 1 — Microsoft side: register the app and consent the read-only scopes . . . 12

PART 2 — ServiceNow side: install and connect . . . 14

PART 3 — Map the CMDB model, reconcile, and schedule . . . 14
Alternative (no ITOM entitlement): custom Integration Hub import . . . 16
5. Testing & Validation — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync) 16
Test 1 — Microsoft-side preflight (auth + scopes + read) . . . 16
Test 2 — ServiceNow connection test . . . 16
Test 3 — First import lands devices (pilot) . . . 17
Test 4 — A known device is correct and NOT duplicated . . . 17
Test 5 — Reconciliation against the Microsoft source of truth . . . 17
Test 6 — No duplicates vs the existing Workspace ONE feed . . . 17
Test 7 — Ownership / compliance signal is usable . . . 17
Test 8 — App inventory (only if the app stream is enabled) . . . 18
Test 9 — Scheduled recurrence . . . 18
Test 10 — Coverage matches expectation before cutover . . . 18
6. Troubleshooting — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync) 18
Authentication & authorization . . . 19
Throttling & performance . . . 19
Reconciliation & CMDB data quality . . . 20
Workspace ONE transition . . . 20
Government / sovereign cloud . . . 21
7. Best Practices — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync) 21
Identity & credentials . . . 21
Reconciliation & CMDB hygiene . . . 22
Scope & scheduling . . . 22
Workspace ONE transition . . . 22
Operations . . . 22
Government / sovereign cloud . . . 23
8. Deliverables Summary — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync) 23
What you receive . . . 23
Key verified facts (with the sources relied on) . . . 24
Out of scope (deferred, referenced not reproduced) . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync)

The situation

The organization is moving endpoint management from **Workspace ONE (AirWatch) to Microsoft Intune**. Today, ServiceNow's **CMDB** is fed device and app inventory by the Workspace ONE data feed — the data the service desk, incident, change, and asset processes depend on. When Workspace ONE goes away, that feed goes with it. **This solution re-establishes the same CMDB device-inventory feed from Intune**, so the CMDB never goes dark and every device stays reconciled to one record.

What this delivers

A repeatable, self-guided implementation that connects **ServiceNow to Microsoft Intune** so ServiceNow **collects managed-device (and installed-app) inventory into the CMDB on a schedule**:

- **Microsoft side:** a dedicated, least-privilege **Entra app registration** with **read-only**

Microsoft Graph device-management permissions (admin-consented) that ServiceNow authenticates to.

- **ServiceNow side:** the **Service Graph Connector for Microsoft Intune** (or a custom Integration

Hub import where ITOM isn't licensed) configured, mapped to the correct **CMDB hardware classes**, and reconciled through **IRE on serial number** so Intune devices **update existing CIs** — including the current Workspace ONE records — instead of creating duplicates.

- **Cutover:** run the Intune feed in parallel, validate coverage, then retire the Workspace ONE feed.

Why it matters

- **Continuity through the migration** — the CMDB inventory feed survives the WS1 → Intune change.
- **One reconciled record per device** — deduplicated across every discovery source, keyed on serial.
- **Faster incidents** — device, owner, compliance, and apps are already on the ticket.
- **Secure by design** — read-only, scoped, admin-consented Graph access; ServiceNow changes nothing in

Intune.

What it is not (scope discipline)

This is an **inventory data feed**, not a management integration and not a licensing project. The **device migration** (WS1PROG), the **SAM license workflow** (SNOWSAM), and **software entitlement reconciliation** (SWDISC) are deferred to their dedicated solutions and referenced, not reproduced.

Approach

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — validated with included PowerShell scripts (preflight, Intune export, sync/reconciliation comparison, coverage report, and a guarded credential

remediation) and explicit pass/fail gates. **Delivered as click-by-click documentation plus tested scripts** — implementable by an in-house administrator at a fraction of consulting cost.

> ***Accuracy note:*** *ServiceNow product/app, plugin, table, and field names change across releases; the guide flags each release-specific name to* ***verify against current ServiceNow documentation****. The Microsoft Graph scopes are verified against Microsoft Learn.*

1. Solution Overview — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync)

What this integration is

Microsoft Intune is the source of truth for who owns which managed device, its hardware, its compliance state, and the apps installed on it. The ServiceNow **CMDB** is the source of truth the service desk, change, incident, and asset processes run against. This solution connects the two so **ServiceNow collects the Intune managed-device (and installed-app) inventory into the CMDB on a schedule** — the Intune equivalent of the Workspace ONE (AirWatch) MDM feed many organizations run today. When a technician opens an incident, the affected device, its owner, its compliance, and its apps are already in ServiceNow, reconciled against every other discovery source.

This is an **inventory / data-feed** integration (Intune → ServiceNow CMDB). It is **not** a management integration — ServiceNow reads Intune; it does not push policy back to Intune.

How it works (the data flow)

1. ServiceNow authenticates to **Microsoft Graph** as a registered **Microsoft Entra application**

(client-credentials OAuth 2.0 — Client ID, Client Secret, Tenant ID).

1. On a schedule, the connector's **data streams** call Graph and pull:

• **Managed devices** — …`/deviceManagement/managedDevices` (hardware, OS, ownership, compliance,

serial number, last check-in).

• **Detected / installed applications** — …`/deviceManagement/detectedApps` (the app inventory per

device).

1. The payload is staged and passed through ServiceNow's **Identification and Reconciliation Engine

(IRE)**, which** matches each Intune device to an existing CI **(by identifier — typically serial number) or creates a new one, so records are** deduplicated**, not duplicated.

1. Devices land in the CMDB **hardware classes** — laptops/desktops in `cmdb_ci_computer` and mobile

devices in the handheld/mobile class — with ServiceNow **Intune** recorded as the discovery source.

1. Downstream ServiceNow processes (Incident, Problem, Change, Request, Asset) now see a current,

reconciled device record with its owner and compliance.

> ***Verify the exact connector data-stream names and CMDB target classes against the current ServiceNow product documentation for your release*** *— ServiceNow has renamed and reworked these across releases (see §2 and §3). The functional model above is stable; the object names drift.*

The supported product path

The current, supported way ServiceNow ingests Intune managed-device data is the **Service Graph Connector for Microsoft Intune** (published on the ServiceNow Store). It is built on ServiceNow's **Integration Hub / Flow Designer** data-stream framework and writes through **IRE** into the CMDB. It requires an **ITOM Visibility or ITOM Discovery** subscription (see §2). Where an organization cannot license the Store connector, the same Graph endpoints and CMDB tables can be driven with a **custom Integration Hub / REST + Transform Map** import — the Microsoft-side setup and the CMDB target model in this guide apply to both. **Confirm the current product name, packaging, and prerequisites on the ServiceNow Store / docs for your instance release before you buy or build.**

Replacing the Workspace ONE (AirWatch) feed

Organizations moving from Workspace ONE to Intune already feed the CMDB from the **Service Graph Connector for VMware Workspace ONE UEM** (the same Service Graph framework, same CMDB hardware classes). This solution **re-establishes that same data feed from Intune** so the CMDB never goes dark during the change:

- Stand up the **Intune** connector **in parallel** with the existing Workspace ONE feed.
- Because both feeds identify devices through **IRE on the same identifier (serial number)**, an Intune record **reconciles onto the existing Workspace ONE CI** rather than creating a duplicate.
- Once the Intune feed is validated and coverage matches, **retire the Workspace ONE connector's schedule** — the CI survives; only its discovery source changes.

What data flows in

- **Device identity & hardware** — name, manufacturer, model, serial number, OS + version.
- **Ownership** — the Intune **primary user** / enrolled user, and corporate vs. personal ownership.
- **Compliance** — Intune compliance state and last check-in / last sync time.
- **Installed / detected applications** — the per-device app inventory from Intune.

Benefits

- **One reconciled inventory** — Intune devices join every other discovery source under one CI per device, deduplicated by IRE.
- **Faster incident handling** — the device, owner, compliance, and apps are already on the ticket.
- **Continuity through the WS1 → Intune change** — the CMDB feed is re-established, not lost.
- **Least-privilege, read-only** — ServiceNow reads Intune through scoped, admin-consented Graph permissions; it changes nothing in Intune.

Limitations (be honest with buyers)

• **Inventory only.** ServiceNow reads Intune device/app data; it does not manage Intune or push policy. Remote actions still happen in Intune.

• **Store connector needs ITOM.** The Service Graph Connector requires an **ITOM Visibility or Discovery** subscription; without it you use the custom Integration Hub import path.

• **Only enrolled devices appear.** A device that never enrolled in Intune is not in the Graph feed and will not appear in the CMDB until it enrolls.

• **App inventory ≠ license position.** The detected-app feed lists what is installed; it is **not** a software-entitlement or license-compliance calculation (see Out of scope).

• **Sovereign clouds differ.** US Gov (GCC High / DoD) tenants use different Graph and sign-in endpoints — see the Government note in §2/§3.

Out of scope (deferred to dedicated solutions — referenced, not reproduced)

• **ServiceNow Software Asset Management (SAM) license workflow** — publisher/license models, entitlements, true-up, reclamation. Deferred to the dedicated **SNOWSAM** solution.

• **Software entitlement reconciliation** — matching installed apps to purchased entitlements and computing an effective license position. Deferred to the dedicated **SWDISC** solution.

• **The Workspace ONE → Intune device migration** itself (un-enrolling from WS1, enrolling into Intune, profile/app parity). Deferred to the **WS1PROG** Workspace ONE migration program. This solution re-establishes the **CMDB data feed** after that migration; it does not perform the device migration.

The standard deployment model

> ***Register the Entra app + consent the read-only Graph scopes → configure the ServiceNow connection → map the field/CMDB model and IRE identifier → schedule the import → assign a pilot → validate devices land and reconcile → expand.*** *repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand.***

2. Requirements Guide — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide Intune prerequisites are in `global-requirements-intune-solutions.md`. This is a **two-platform** integration, so the requirements are grouped: **Microsoft (Intune / Entra / Graph)**, then **ServiceNow**.

> ***Accuracy note:*** *ServiceNow product names, plugin/store-app packaging, and CMDB class names change across releases. Where this guide names a specific ServiceNow app, plugin, table, or field,* ***confirm it against the ServiceNow Store / docs for your instance release*** *before relying on it. The Microsoft Graph scope names below are verified against Microsoft Learn (current at authoring).*

Part A — Microsoft (Intune / Entra / Graph)

2.1 Microsoft Intune with devices enrolled

Description: An Intune-bearing subscription with the target devices enrolled and reporting. **Why:** ServiceNow can only import devices Intune actually knows about; unenrolled devices are not in the Graph feed. **Verify:** `https://intune.microsoft.com` → **Devices** → **All devices** shows the fleet; each device has a recent **last check-in**. **Implement:** Assign Intune (M365 E3/E5, Business Premium, or Intune Plan 1); enable automatic enrollment and enroll devices (see the Device Enrollment solution).

2.2 Microsoft Entra application registration (for ServiceNow to authenticate)

Description: A dedicated Entra **app registration** ServiceNow uses to call Microsoft Graph — named `SVC – ServiceNow CMDB – Intune Read` (use one clear, auditable name). **Why:** ServiceNow authenticates to Graph with this app's **Client ID + Client Secret + Tenant ID** (client-credentials OAuth 2.0). A dedicated app keeps the integration auditable and independently revocable. **Verify:** Entra admin center → **Identity** → **Applications** → **App registrations**; the app exists and its **Application (client) ID** and **Directory (tenant) ID** are recorded. **Implement:** Entra → **App registrations** → **New registration** → name `SVC – ServiceNow CMDB – Intune Read`, single-tenant → **Register** (full click-path in §3, Step 1).

2.3 Client secret (or certificate) on the app registration

Description: A client secret (or certificate credential) on the app above, with a tracked expiry. **Why:** It is the credential ServiceNow presents to Graph. An expired secret silently breaks the feed. **Verify:** App registration → **Certificates & secrets**; a valid, unexpired secret/cert exists and its **expiry date is recorded** for renewal. **Implement: Certificates & secrets** → **New client secret**, description `ServiceNow CMDB feed`, choose a defined lifetime (e.g. 12–24 months), copy the **Value** once. Set a renewal reminder.

2.4 Microsoft Graph application permissions (least privilege, read-only) + admin consent

Description: The app registration must hold these **Application** (app-only) Graph permissions, **admin-consented**:

- `DeviceManagementManagedDevices.Read.All` — read managed devices (device, category, **detected

app**, malware info). *This is the core inventory scope.*

- `DeviceManagementApps.Read.All` — read Intune app inventory (client apps / app config).
- `DeviceManagementServiceConfig.Read.All` — read enrollment/service configuration context.
- `Directory.Read.All` — resolve users/ownership for the device→owner association.

Why: These are the exact scopes the ServiceNow Intune connector uses to read device + app inventory. All are **read-only**; none permit changing Intune. Least-privilege rule: **do not** grant `…ReadWrite.All` or

…`PrivilegedOperations.All` — a CMDB feed never needs write. **Verify:** App registration → **API permissions**; the four Application permissions are listed with **"Granted for <tenant>"** (green check). Confirm they are **Application**, not Delegated. **Implement: API permissions** → **Add a permission** → **Microsoft Graph** → **Application permissions**, add the four scopes, then **Grant admin consent for <tenant>**. (Run `SNOWINT-Preflight.ps1` to confirm.)

> ***Least-privilege guidance:*** *the minimum for device inventory alone is* `DeviceManagementManagedDevices.Read.All`*. Add* `DeviceManagementApps.Read.All` *only if you import the app inventory, and* `Directory.Read.All` *only to resolve owner/user detail. Grant the smallest set your connector configuration actually reads.*

2.5 Global / Privileged Role Administrator to grant consent

Description: An admin who can grant tenant-wide admin consent for application Graph permissions. **Why:** Application permissions require admin consent before Graph will honor them. **Verify:** Entra → **Roles and administrators**; you hold **Global Administrator** or **Privileged Role Administrator** (Cloud Application Administrator can grant consent for many app permissions). **Implement:** Have a Global Administrator perform the **Grant admin consent** click in Step 2.4.

2.6 Network egress to Microsoft Graph from ServiceNow's caller

Description: Outbound HTTPS from wherever the connection is made (ServiceNow instance directly, or a **MID Server** if your design routes Graph calls through one) to the Graph endpoint. **Why:** No egress to Graph = no feed. **Verify:** From the caller, reach `https://graph.microsoft.com` (Commercial) — or the sovereign endpoint below for Gov. If a MID Server is used, confirm it is **Up** in ServiceNow. **Implement:** Allow outbound 443 to `graph.microsoft.com` and `login.microsoftonline.com` (Commercial) through the proxy/firewall; if using a MID Server, install/validate it first.

Part B — ServiceNow

2.7 Supported ServiceNow release + CMDB

Description: A supported ServiceNow release (e.g. current N/N-1 family) with the CMDB in use. **Why:** Connector packaging and CMDB identifier rules differ by release; the CMDB must exist as the target of record. **Verify:** ServiceNow → **System Diagnostics** → **Stats** (or the instance banner) shows the release; the CMDB (**Configuration** → **All**) is populated. **Implement:** Upgrade to a supported release; ensure CMDB is in use.

2.8 ITOM Visibility or ITOM Discovery subscription (for the Service Graph Connector)

Description: The **Service Graph Connector for Microsoft Intune** requires a Subscription-Unit-based **ITOM Visibility** or **ITOM Discovery** subscription. **Why:** The Store connector and its Service Graph data-stream framework are entitled under ITOM. **Verify:** Confirm the ITOM Visibility/Discovery entitlement with your ServiceNow account team, and that the connector installs from the Store. **Implement:** License ITOM Visibility or Discovery; **or** use the custom Integration Hub / REST + Transform Map import path (which does not require the Store connector) — see §3. **Verify against current ServiceNow licensing for your release.**

2.9 Service Graph Connector for Microsoft Intune (+ its dependencies) installed

Description: The Store app **"Service Graph Connector for Microsoft Intune"** and its dependencies (Integration Hub / Flow Designer, Integration Commons for CMDB / the Service Graph framework, and the CMDB CI Class Models) installed and active. **Why:** These deliver the data streams, the robust transform, and IRE write-through. **Verify: System Applications** → **All Available Applications** → **All** (or **Store** → **Downloads**); the connector and its dependencies show **Installed**. **Implement:** Install the connector from the ServiceNow Store; accept/plugin-activate its dependencies. **Confirm the exact app name, version, and dependency list on the Store for your release** (names have changed across releases).

2.10 ServiceNow roles to configure the connector

Description: An admin who can create connection & credential aliases, configure the data source, and run the import. **Why:** Configuring an Integration Hub connector and CMDB transform requires elevated ServiceNow roles. **Verify:** You hold the roles the connector documents (commonly `admin`, plus discovery/integration admin roles such as `sn_disco.*` / `integration_hub_admin`). **Implement:** Have a ServiceNow administrator assign the required roles. **Verify the exact role names in the connector's docs for your release.**

2.11 CMDB Identification and Reconciliation (IRE) identifier for devices

Description: CMDB **identifier rules** for the hardware classes keyed on a stable identifier — typically **serial number** — so Intune records dedupe against existing CIs (including the current Workspace ONE feed's CIs). **Why:** Without a matching identifier, the Intune feed would **create duplicate CIs** instead of updating the existing device. **Verify: Configuration** → **CI Class Manager** → **(Hardware / Computer / Handheld class)** → **Identification/Reconciliation**; a serial-number-based identifier rule is active. **Implement:** Ensure the out-of-box hardware identifier (serial number) is enabled and that both the Intune and Workspace ONE feeds are registered as **discovery sources** so reconciliation, not duplication, occurs. **Verify the exact class names + identifier entries for your release.**

Government / sovereign cloud (GCC High / DoD) — read before configuring a Gov tenant

US Government tenants do **not** use the Commercial endpoints. The **integration model is identical**; only the endpoints change:

- **Graph endpoint:** `https://graph.microsoft.us` (GCC High / GCC) or `https://dod-graph.microsoft.us`

(DoD) — **not** `graph.microsoft.com`.

- **Sign-in authority:** `https://login.microsoftonline.us` — not `login.microsoftonline.com`.
- **Intune admin portal:** `https://intune.microsoft.us` (Manage endpoint on the US Gov cloud).

Configure the ServiceNow connection's OAuth token URL and Graph base URL to the `.us` **sovereign endpoints**, and confirm the Store connector supports an overridable/sovereign endpoint (some connector versions assume the Commercial Graph host). **Verify for GCC High** whether your connector release exposes a configurable Graph/authority endpoint; if not, the custom Integration Hub import (with the `.us` endpoints) is the fallback. The PowerShell validation scripts in this package are cloud-aware — run them with `-Cloud USGov` (or `USGovDoD`).

Checklist

Microsoft

- Intune subscription; target devices enrolled and checking in
- Entra app registration `SVC – ServiceNow CMDB – Intune Read` (client ID + tenant ID recorded)
- Client secret/cert created; **expiry date recorded**
- Graph **Application** permissions granted + **admin-consented** — `DeviceManagementManagedDevices.Read.All`, `DeviceManagementApps.Read.All`, `DeviceManagementServiceConfig.Read.All`, `Directory.Read.All`
- Egress to Graph (`graph.microsoft.com` / `graph.microsoft.us` for Gov)
- `SNOWINT-Preflight.ps1` passes

ServiceNow

- Supported release; CMDB in use
- ITOM Visibility or Discovery subscription (or custom-import path chosen)
- Service Graph Connector for Microsoft Intune + dependencies installed
- Configuration roles assigned
- Serial-number IRE identifier active; Intune + Workspace ONE registered as discovery sources

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide

changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync)

Follow these steps **in order**. You work in two consoles: the **Microsoft Entra admin center** (`https://entra.microsoft.com`) and **Intune** (`https://intune.microsoft.com`) on the Microsoft side, and your **ServiceNow instance** on the ServiceNow side.

> ***Model:*** *Entra app registration & consent → ServiceNow connection → field/CMDB mapping & IRE → scheduled import → validate.*
>
> ***Names drift — verify as you go.*** *The Microsoft-side steps and Graph scopes are stable and verified. On the ServiceNow side, the exact **menu paths, connector app name, data-stream names, and CMDB class/field names change across releases** — confirm each against the connector's docs for **your** instance release. Where a name is release-specific it is flagged inline.*

PART 1 — Microsoft side: register the app and consent the read-only scopes

Step 1 — Register the Entra application ServiceNow will use

1. Go to `https://entra.microsoft.com` → **Identity** → **Applications** → **App registrations**.
2. Click **New registration**.
3. **Name:** `SVC – ServiceNow CMDB – Intune Read`.
4. **Supported account types: Accounts in this organizational directory only (single tenant)**.
5. Leave **Redirect URI** blank (this is a client-credentials, app-only integration). Click **Register**.
6. On the app **Overview**, copy and record the **Application (client) ID** and the **Directory (tenant) ID** — ServiceNow needs both.

Step 2 — Create the client secret

1. In the app, open **Certificates & secrets** → **Client secrets** → **New client secret**.
2. **Description:** `ServiceNow CMDB feed`. **Expires:** choose a defined lifetime (e.g. 24 months). Click **Add**.
1. **Copy the secret Value immediately** (it is shown only once) and store it in your secret vault.

Record the expiry date for renewal. *(A certificate credential is a stronger alternative to a secret — use one if your ServiceNow connection supports certificate auth.)*

Step 3 — Add the least-privilege Graph Application permissions

1. In the app, open **API permissions** → **Add a permission** → **Microsoft Graph** → **Application permissions**.
2. Add each of these (search by name, tick, **Add permissions**):

- `DeviceManagementManagedDevices.Read.All`
- `DeviceManagementApps.Read.All`
- `DeviceManagementServiceConfig.Read.All`
- `Directory.Read.All`

1. Confirm each row shows **Application** (not Delegated). Remove any unused default delegated permission you do not need. **Do not** add any `…ReadWrite.All` or `…PrivilegedOperations.All` scope — this feed is read-only.

Step 4 — Grant admin consent

1. Still on **API permissions**, click **Grant admin consent for <tenant>** → **Yes**.
2. Confirm every permission row shows **"Granted for <tenant>"** with a green check.
3. Validate from a Microsoft-side workstation: run `SNOWINT-Preflight.ps1` (Commercial) — it confirms the app can authenticate, the four scopes are consented, and managed devices are readable. For a Gov tenant run `SNOWINT-Preflight.ps1 -Cloud USGov`.

> ***Government (GCC High / DoD):*** *perform Steps 1–4 in* `https://entra.microsoft.com` *for your Gov tenant, but note the ServiceNow connection (Part 2) must target* `https://graph.microsoft.us` *and* `https://login.microsoftonline.us`*. See §2*

*Government note. **Verify for GCC High** that your connector release supports the sovereign endpoints.*

PART 2 — ServiceNow side: install and connect

Step 5 — Install the connector and its dependencies

1. In ServiceNow, open the **Store** (or **System Applications** → **All Available Applications** → **All**).

2. Find **"Service Graph Connector for Microsoft Intune"**, click **Install/Update**, and install its dependencies (Integration Hub / Flow Designer, Integration Commons for CMDB / Service Graph framework, CMDB CI Class Models).

1. Confirm the connector and dependencies show **Installed / Active**.

(No ITOM entitlement? Skip to the custom-import alternative at the end of Part 3.) **Confirm the exact app name, version, and dependency list on the Store for your release.**

Step 6 — Create the connection & credential alias to Microsoft Graph

1. Open the connector's guided setup (commonly under **Service Graph Connectors** or the connector's application menu) → the **Microsoft Intune** connection.

1. Enter the values from Part 1:

• **Tenant ID** = Directory (tenant) ID (Step 1).

• **Client ID** = Application (client) ID (Step 1).

• **Client Secret** = the secret Value (Step 2).

• **Graph / token endpoint** = Commercial (`graph.microsoft.com` / `login.microsoftonline.com`) — **or** the `.us` sovereign endpoints for a Gov tenant.

1. Save. Use the connection's **Test** action to confirm authentication succeeds. If your design uses a **MID Server** for egress, select it here and confirm it is **Up**. **Verify the exact field labels and connection/credential-alias flow in the connector docs for your release.**

PART 3 — Map the CMDB model, reconcile, and schedule

Step 7 — Confirm the data streams and CMDB target classes

1. In the connector's configuration, review the enabled **data streams** and their CMDB targets:

• **Managed devices** → the **computer** class `cmdb_ci_computer` (Windows/macOS laptops/desktops) and the **handheld/mobile** class (iOS/Android) — confirm the exact handheld class name for your release (e.g. `cmdb_ci_handheld_computing`).

• **Detected / installed applications** → the software/installed-software model — **confirm the exact target table for your release**.

1. Enable only the streams you want (device inventory is the core; enable the app stream only if you

will consume the app inventory). **The CMDB class names above are release-specific — verify each in CI Class Manager / the connector docs before relying on them.**

Step 8 — Verify/adjust the field mapping and IRE identifier

1. Review the connector's **robust transform / field map** so Intune fields land on the right CI

fields: **serial number**, name, manufacturer, model, OS + version, **assigned/primary user** (ownership), **compliance state**, and **last check-in**.

1. Confirm **Identification and Reconciliation (IRE)** matches on a stable identifier —

serial number — so each Intune device **updates the existing CI** rather than creating a duplicate. **Configuration** → **CI Class Manager** → **(class)** → **Identification/Reconciliation.**

1. **Reconcile against the existing Workspace ONE feed:** ensure both **Microsoft Intune** and

Workspace ONE are registered **discovery sources** and that the serial-number identifier is the same for both, so an Intune record lands on the **same CI** the WS1 feed already created — no duplicate device.

Step 9 — Schedule the import

1. In the connector, set the **data stream schedule** to run on a cadence (e.g. **daily**). Enable it.
2. For the first run, trigger the import **manually** to validate before leaving it on the schedule.

Verify the exact scheduling control name for your release.

Step 10 — Pilot: run once and validate a known device

1. Run the import (manual trigger). Wait for it to complete.
2. On the **Microsoft** side, export the same dataset with `SNOWINT-Export-ManagedDevices.ps1` →

a CSV of Intune managed devices (the exact set ServiceNow should import) for reconciliation.

1. In ServiceNow, open the CMDB and find a **known pilot device by serial number**. Confirm:

• It exists as **one** CI (no duplicate), in the correct hardware class.

• Owner, OS, compliance, and last check-in are populated and correct.

• Its **discovery source** includes **Intune**.

1. Run `SNOWINT-Validate-Sync.ps1 -CmdbCsv <export from ServiceNow>` to compare the Intune device

set against a CMDB export (serials present/missing/duplicated). Complete the **Testing Guide**.

Step 11 — Expand to production (and retire the Workspace ONE feed)

1. Widen the import scope from the pilot set to the full fleet; let it run on the daily schedule.
2. Run `SNOWINT-Report-Coverage.ps1` to confirm managed-device coverage by platform/ownership

matches expectations wave over wave.

1. Once the Intune feed is validated and coverage matches the Workspace ONE feed, **retire the

Workspace ONE connector's schedule** — the CIs persist and reconcile onto the Intune source; only the discovery source changes. *(The device migration itself is the WS1PROG program — out of scope here.)*

Alternative (no ITOM entitlement): custom Integration Hub import

If you cannot license the Store connector, build the same feed with native ServiceNow tooling:

1. Create an **OAuth 2.0 (client-credentials) application registry** entry in ServiceNow pointing at

the Entra app from Part 1, scope `https://graph.microsoft.com/.default` (or the `.us` equivalent).

1. Create **REST messages / a Data Source** that GET …`/deviceManagement/managedDevices` (and

…`/deviceManagement/detectedApps`), handling paging (`@odata.nextLink`).

1. Land the JSON via **Transform Maps** into `cmdb_ci_computer` / the handheld class, **coalescing on

serial number **(or route through the** IRE API** so reconciliation matches the Store connector).

1. Schedule the import (Scheduled Data Import / Scheduled Job).

This path uses the **same Microsoft setup and the same CMDB target model** as the connector.

Order of operations

`Entra app registration → client secret → read-only Graph scopes → admin consent → (Preflight) → install connector → connection to Graph → data streams + CMDB classes → field map + serial-number IRE → reconcile vs Workspace ONE → schedule → pilot import → validate → expand → retire WS1 feed`

5. Testing & Validation — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync)

Work top to bottom. Each test has an explicit **PASS / FAIL** gate. Do not expand to production until the pilot tests pass.

Test 1 — Microsoft-side preflight (auth + scopes + read)

1. Run `SNOWINT-Preflight.ps1` (add `-Cloud USGov` for a Gov tenant).

• **PASS:** connects to Graph; the four expected scopes are present; managed devices are readable

(sample count > 0).

• **FAIL:** any scope missing / not consented, or the managed-device read errors. Fix the app

registration / admin consent (§3 Steps 3–4) before touching ServiceNow.

Test 2 — ServiceNow connection test

1. In the connector's connection, run the built-in **Test** action.

• **PASS:** authentication to Graph succeeds (token acquired). If a MID Server is used, it is **Up**.

• **FAIL:** auth error → recheck Tenant ID / Client ID / Client Secret and the Graph/token endpoint (Commercial vs `.us`).

Test 3 — First import lands devices (pilot)

1. Trigger the import manually (§3 Step 10). Wait for completion; check the import/data-stream log for errors.

• **PASS:** the run completes; devices are created/updated in the CMDB with no transform errors.

• **FAIL:** transform/IRE errors in the log → see Troubleshooting (mapping / identifier).

Test 4 — A known device is correct and NOT duplicated

1. Pick a known pilot device. In ServiceNow, search the CMDB by its **serial number**.

• **PASS:** exactly **one** CI in the correct hardware class; owner, OS, compliance, and last check-in are populated and match Intune; **discovery source** includes **Intune**.

• **FAIL:** two CIs for one serial → IRE identifier problem (see Test 6); wrong/blank fields → field map problem.

Test 5 — Reconciliation against the Microsoft source of truth

1. Export the Intune set with `SNOWINT-Export-ManagedDevices.ps1`.

2. Export the CMDB device list to CSV (serial column), then run `SNOWINT-Validate-Sync.ps1 -CmdbCsv <cmdb.csv>`.

• **PASS: Matched** ≈ the pilot count; **Missing in CMDB** ≈ 0 (allowing for very recent enrollments); **DUPLICATE serials = 0**.

• **FAIL:** significant Missing (import scope/schedule) or any Duplicates (identifier) — resolve before expanding.

Test 6 — No duplicates vs the existing Workspace ONE feed

1. For a device present in **both** the Workspace ONE feed and Intune, confirm in the CMDB there is **one** CI whose **discovery sources list both** Workspace ONE and Intune.

• **PASS:** single reconciled CI, both sources listed.

• **FAIL:** two CIs (one per source) → the serial-number IRE identifier is not shared across sources; fix the identification rule (§3 Step 8) and re-run.

Test 7 — Ownership / compliance signal is usable

1. On several CIs, confirm the **assigned/primary user** and **compliance state** populated from

Intune, and that the fields are visible on an Incident form's affected-CI lookup.

- **PASS:** owner + compliance present and visible to the service desk.
- **FAIL:** blank owner → confirm `Directory.Read.All` is consented and the owner field is mapped.

Test 8 — App inventory (only if the app stream is enabled)

1. On a known device, confirm its installed/detected applications appear in the CMDB software model.

- **PASS:** the device's apps are listed.
- **FAIL:** confirm `DeviceManagementApps.Read.All` is consented and the detected-apps stream is enabled and mapped. *(Software licensing/entitlement is out of scope — SNOWSAM/SWDISC.)*

Test 9 — Scheduled recurrence

1. Confirm the data stream is scheduled (e.g. daily) and enabled; after the next scheduled run, confirm **last check-in / last updated** on a CI advanced.

- **PASS:** the CI updates on schedule without manual triggering.
- **FAIL:** schedule disabled or erroring → re-enable and check the job log.

Test 10 — Coverage matches expectation before cutover

1. Run `SNOWINT-Report-Coverage.ps1`; compare platform/ownership totals to the CMDB count.

- **PASS:** CMDB device count ≈ Intune managed-device count (minus intentionally out-of-scope platforms); coverage matches or exceeds the Workspace ONE feed before you retire it.
- **FAIL:** large gap → widen import scope / investigate missing serials before retiring WS1.

Pilot exit criteria (all must be true)

- Preflight + connection tests PASS
- Known device correct, single CI, Intune as a discovery source
- `SNOWINT-Validate-Sync.ps1`: 0 duplicates, ~0 missing
- Dual-source device reconciles to one CI (no WS1/Intune duplicate)
- Owner + compliance populated and visible to the service desk
- Scheduled run updates CIs automatically
- Coverage report matches expectation

6. Troubleshooting — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync)

Work from the symptom. Each entry: **Symptom** → **Likely cause** → **Fix.** Most feed failures are authentication (expired secret / missing consent) or reconciliation (identifier) problems.

Authentication & authorization

The connection test fails / the feed stopped pulling data

- **Likely cause:** the app registration's **client secret expired**, or the wrong Tenant/Client ID or endpoint is configured.
- **Fix:** run `SNOWINT-Remediation.ps1 -AppId <client-id>` to check secret expiry; if expired, re-run with `-Apply` to mint a new secret, paste it into the ServiceNow connection, and **Test**. Confirm Tenant ID / Client ID and that the Graph/token endpoint matches the cloud (Commercial vs `.us`). Remove the old secret after ServiceNow is updated.

403 Forbidden / "Application is not authorized"

- **Likely cause:** the required Graph **Application** permission is missing or **admin consent** was not granted (or a scope was granted as Delegated).
- **Fix:** Entra → app → **API permissions**; confirm the four Application scopes show **"Granted for <tenant>"**. Click** Grant admin consent**. Re-run `SNOWINT-Preflight.ps1` to confirm, then re-test the ServiceNow connection.

Auth works but 0 devices import

- **Likely cause:** no devices enrolled in this tenant, the import scope excludes them, or (Gov) the connection points at the wrong sovereign endpoint.
- **Fix:** confirm `SNOWINT-Preflight.ps1` returns a device sample; widen the connector's import scope; for Gov, confirm `graph.microsoft.us` / `login.microsoftonline.us`.

Throttling & performance

Import is slow, times out, or logs HTTP 429 / "TooManyRequests"

- **Likely cause:** Microsoft Graph **throttling** (large fleet + detected-apps stream is heavy), or paging not honored.
- **Fix:** run the import on an **off-peak schedule**; enable only the streams you consume (defer the detected-apps stream if not needed); ensure the connector/import honors `@odata.nextLink` paging and respects `Retry-After`. Reduce frequency — daily is usually sufficient for inventory.

MID Server errors (if used for egress)

- **Likely cause:** MID Server down or blocked from reaching Graph.
- **Fix:** confirm the MID Server is **Up** in ServiceNow and can reach `graph.microsoft.com` / `graph.microsoft.us` on 443 through the proxy/firewall.

Reconciliation & CMDB data quality

Duplicate CIs — two records for one physical device

- **Likely cause:** the **serial-number IRE identifier** is not shared across discovery sources (Intune vs Workspace ONE vs SCCM), so each source creates its own CI. This is the most common CMDB symptom.
- **Fix: Configuration** → **CI Class Manager** → **(hardware class)** → **Identification/Reconciliation**; ensure a **serial-number** identifier rule is active and used by **all** sources; register **Intune** and **Workspace ONE** as discovery sources against the same identifier. Run `SNOWINT-Validate-Sync.ps1` — duplicates should drop to 0. De-dupe existing duplicates per your CMDB remediation process.

Devices with no serial number don't reconcile

- **Likely cause:** some devices (certain VMs / mobile hardware) report an empty or non-unique serial.
- **Fix:** run `SNOWINT-Report-Coverage.ps1` to count no-serial devices; add a secondary identifier (e.g. Intune device ID / Entra device ID) for those classes, or handle them out-of-band. Don't weaken the serial identifier for the whole fleet.

Wrong or blank owner / compliance on the CI

- **Likely cause:** field mapping incomplete, or `Directory.Read.All` not consented (owner lookup fails).
- **Fix:** confirm the connector's field map populates assigned/primary user and compliance; confirm `Directory.Read.All` is consented (`SNOWINT-Preflight.ps1`). Decide field authority so another source isn't overwriting Intune's compliance/last-check-in.

Stale records — CIs stop updating

- **Likely cause:** the data-stream **schedule is paused/erroring**, the secret expired, or consent was revoked.
- **Fix:** confirm the schedule is enabled and its last run succeeded; check the import log; run `SNOWINT-Remediation.ps1` for secret/scope health. Treat a stalled last-updated as an incident.

Devices in Intune are missing from the CMDB

- **Likely cause:** import scope too narrow, first scheduled run hasn't occurred, or transform errors.
- **Fix:** run `SNOWINT-Validate-Sync.ps1` to list missing serials; widen scope / trigger a manual run; check the transform log for row-level errors.

Workspace ONE transition

After enabling Intune, device counts double

- **Likely cause:** the Intune feed created new CIs instead of reconciling onto the Workspace ONE CIs.
- **Fix:** as above — shared serial-number identifier across both sources. Do **not** retire the

Workspace ONE feed until `SNOWINT-Validate-Sync.ps1` shows 0 duplicates and matching coverage.

Retired the WS1 feed and lost devices from the CMDB

• **Likely cause:** those devices had not yet re-enrolled in Intune (still WS1-only) when the feed was

cut, so nothing refreshed them.

• **Fix:** confirm full Intune enrollment coverage (`SNOWINT-Report-Coverage.ps1`) **before**

retiring the WS1 feed; the device migration itself is the WS1PROG program.

Government / sovereign cloud

Gov tenant: connection or scripts fail against `graph.microsoft.com`

• **Likely cause:** Commercial endpoints used for a GCC High / DoD tenant.

• **Fix:** point the ServiceNow connection at `https://graph.microsoft.us` (or `dod-graph.microsoft.us`)

and `https://login.microsoftonline.us`; run scripts with `-Cloud USGov` / `USGovDoD`. **Verify for GCC High** that your connector release exposes a configurable sovereign endpoint; if not, use the custom Integration Hub import path with the `.us` endpoints.

Escalation

If a symptom persists after these steps, capture: the ServiceNow import/data-stream log, the connection test result, `SNOWINT-Preflight.ps1` output, and `SNOWINT-Validate-Sync.ps1` output — and consult the current **ServiceNow Service Graph Connector for Microsoft Intune** documentation for your release (names/behavior are release-specific) alongside Microsoft Graph device-management docs.

7. Best Practices — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync)

Identity & credentials

• **Dedicated, named app registration.** Use one app (`SVC – ServiceNow CMDB – Intune Read`) for this

feed only — never a shared or personal app — so it is auditable and independently revocable.

• **Least privilege, read-only.** Grant only the four read scopes the feed needs. Never grant

…`ReadWrite.All` or …`PrivilegedOperations.All`. A CMDB feed must not be able to change Intune.

• **Track secret expiry.** A silently expired client secret is the #1 cause of a dead feed. Record the

expiry, set a renewal reminder, and prefer a **certificate** credential where the connection supports it. Rotate additively (new secret → update ServiceNow → remove old).

• **Store the secret in a vault**, not in a ticket, wiki, or script.

Reconciliation & CMDB hygiene

• **Serial number is the anchor.** Make serial number the shared IRE identifier across **all** device discovery sources (Intune, Workspace ONE, SCCM, Discovery) so devices reconcile onto one CI. Devices with no serial cannot reconcile by serial — surface them (coverage report) and handle separately.

• **Register Intune as a discovery source** and keep the out-of-box hardware identifier — don't invent a bespoke identifier that only the Intune feed uses.

• **Let IRE do the matching.** Route the import through IRE (Store connector does this) rather than a naive coalesce, so multi-source reconciliation and CI class precedence behave correctly.

• **Don't overwrite authoritative fields blindly.** Decide, per field, whether Intune or another source is authoritative (e.g. Intune for compliance/last-check-in; procurement/HAM for location/cost) so the feed enriches rather than clobbers.

Scope & scheduling

• **Import only what you'll use.** Enable the device stream first; add the app stream only if the service desk/asset processes consume app inventory — the detected-apps dataset is large.

• **Daily is usually enough.** Match the schedule to how fresh the CMDB must be; more frequent runs cost Graph calls and instance load for little benefit on inventory data.

• **Pilot before fleet.** Always validate a pilot scope (Test 1–8) before widening to the whole fleet.

Workspace ONE transition

• **Run both feeds in parallel**, reconcile on the shared serial identifier, and only **retire the Workspace ONE schedule** once the Intune feed's coverage matches or exceeds it (coverage report).

• **The CI is the constant.** During cutover the device CI persists; only its discovery source changes from Workspace ONE to Intune. Don't delete and recreate CIs.

• **Keep the boundaries clean:** re-establishing the *feed* (this solution) is separate from the *device migration* (WS1PROG) and from *license reconciliation* (SNOWSAM/SWDISC).

Operations

• **Watch the import/data-stream logs** for transform and IRE errors after each run; a rising error count is an early warning.

• **Alert on staleness.** If the CMDB last-updated stops advancing, treat it as an incident — usually an expired secret, revoked consent, or a paused schedule.

• **Re-run** `SNOWINT-Validate-Sync.ps1` **periodically** (not just at go-live) to keep duplicates at zero and catch drift.

• **Document the design** — app ID, scopes, connection name, CMDB classes, identifier rule, schedule —

so the next admin can operate it.

Government / sovereign cloud

• Use the `.us` **Graph and sign-in endpoints** end to end for GCC High / DoD; never mix Commercial and sovereign endpoints. Run the scripts with `-Cloud USGov` / `USGovDoD`. **Verify for GCC High** that the connector release supports the sovereign endpoint before committing to the Store connector.

8. Deliverables Summary — ServiceNow ↔ Microsoft Intune Integration (Device Inventory & CMDB Sync)

What you receive

A complete, self-guided implementation package to re-establish the CMDB device-inventory feed from Microsoft Intune into ServiceNow — the Intune equivalent of the Workspace ONE MDM feed.

Documentation (Word + PDF)

1. **Solution Overview** — the data flow (Graph → connector → IRE → CMDB), the supported product path

(Service Graph Connector for Microsoft Intune), and how it replaces the Workspace ONE feed.

1. **Requirements Guide** — two-platform prerequisites (Microsoft: Intune, Entra app registration, least-privilege Graph scopes, admin consent; ServiceNow: ITOM entitlement, connector + dependencies, roles, IRE identifier) — each with Description · Why · Verify · Implement-if-missing.

1. **Implementation Guide** — ordered steps: Entra app registration & consent → ServiceNow connection →

field/CMDB mapping & serial-number IRE → scheduled import → pilot → validate → expand → retire WS1.

1. **Testing & Validation** — 10 tests with explicit PASS/FAIL gates and pilot exit criteria.
2. **Troubleshooting** — auth failures, throttling, duplicate CIs, stale records, and more.
3. **Best Practices** — credentials, reconciliation/CMDB hygiene, scope/scheduling, WS1 transition, ops.

PowerShell scripts (5) — the Microsoft side, read-only by default, cloud-aware

• `SNOWINT-Preflight.ps1` — verify the app registration, consented scopes, and managed-device read.

• `SNOWINT-Export-ManagedDevices.ps1` — export the exact Intune device dataset ServiceNow imports

(for reconciliation/validation).

• `SNOWINT-Validate-Sync.ps1` — compare Intune vs a CMDB export CSV (matched / missing / stale /

duplicate serials).

• `SNOWINT-Report-Coverage.ps1` — managed-device coverage by platform / ownership / compliance /

staleness.

• `SNOWINT-Remediation.ps1` — guarded (`-WhatIf`/diagnose-by-default) fix for a missing/expired

app-registration secret or a missing scope.

All scripts use `Connect-MgGraph -Environment Global|USGov|USGovDoD` so they run in Commercial and US Government clouds.

Key verified facts (with the sources relied on)

• ServiceNow ingests Intune device inventory via the **Service Graph Connector for Microsoft Intune**

(ServiceNow Store), which requires an **ITOM Visibility or Discovery** subscription and writes to the CMDB through IRE.

• The connector authenticates to **Microsoft Graph** with **Application** permissions:

`DeviceManagementManagedDevices.Read.All`, `DeviceManagementApps.Read.All`, `DeviceManagementServiceConfig.Read.All`, `Directory.Read.All` — all read-only, admin-consented.

• Managed devices come from …`/deviceManagement/managedDevices` and detected apps from

…`/deviceManagement/detectedApps`; devices land in the CMDB **computer** class (`cmdb_ci_computer`) and the **handheld/mobile** class.

> *Release-specific ServiceNow names (connector app/version, data-stream names, exact CMDB class/field names, role names, ITOM packaging) **must be verified against current ServiceNow documentation for your instance release** — the guide flags each inline.*

Out of scope (deferred, referenced not reproduced)

• **SNOWSAM** — ServiceNow SAM license workflow.

• **SWDISC** — software entitlement reconciliation / effective license position.

• **WS1PROG** — the Workspace ONE → Intune device migration itself.

Outcome

A scheduled, reconciled, least-privilege Intune → ServiceNow CMDB device-inventory feed that survives the Workspace ONE → Intune transition, keyed on serial number so every device is one trustworthy CI.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SNOWINT-Export-ManagedDevices.ps1

Purpose: Produces a CSV of the same managed-device inventory the ServiceNow Service Graph Connector pulls from Graph (serial number, name, OS, ownership, compliance, last check-in). Use it as the Microsoft-side "source of truth" to reconcile against what actually landed in the CMDB. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SNOWINT-Export-ManagedDevices.ps1 ` -CsvPath C:\Temp\intune-devices.csv [-Cloud Global|USGov|USGovDoD] Requires the Microsoft.Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A CSV (and on-screen count) of all Intune managed devices with the fields the CMDB maps. Hand this to SNOWINT-Validate-Sync.ps1 (or compare to a CMDB export) to prove the feed. .NOTES Intune Solutions — read-only. Cloud-aware (Global / USGov / USGovDoD). > [CmdletBinding()] param([string]$CsvPath = ".\SNOWINT-intune-devices.csv", [ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\SNOWINT-Export-ManagedDevices.ps1 ` -CsvPath C:\Temp\intune-devices.csv [-Cloud Global|USGov|USGovDoD] Requires the Microsoft.Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS A CSV (and on-screen count) of all Intune managed devices with the fields the CMDB maps. Hand this to SNOWINT-Validate-Sync.ps1 (or compare to a CMDB export) to prove the feed. .NOTES Intune Solutions — read-only. Cloud-aware (Global / USGov / USGovDoD). > [CmdletBinding()] param([string]$CsvPath = ".\SNOWINT-intune-devices.csv", [ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

SNOWINT-Preflight.ps1

Purpose: Confirms the Microsoft side of the ServiceNow <-> Intune CMDB feed is ready: that the signed-in identity can reach Graph, that the read-only device-management scopes are present, and that managed devices are actually readable. Run this BEFORE configuring the ServiceNow connection so a broken app registration / missing admin consent is caught early. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SNOWINT-Preflight.ps1 [-Cloud Global|USGov|USGovDoD] Requires the Microsoft.Graph PowerShell SDK. Sign in as an admin who can read device mgmt. (This checks the tenant is ready; the ServiceNow connection itself uses the app's client secret via client-credentials — configure that in ServiceNow, not here.) EXPECTED RESULTS Green PASS lines for: Graph connect, each expected scope, and a managed-device read count. Any red FAIL tells you exactly what to fix (missing scope, no consent, no devices). .NOTES Intune Solutions — read-only. Cloud-aware (Global / USGov / USGovDoD). > [CmdletBinding()] param([ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\SNOWINT-Preflight.ps1 [-Cloud Global|USGov|USGovDoD] Requires the Microsoft.Graph PowerShell SDK. Sign in as an admin who can read device mgmt. (This checks the tenant is ready; the ServiceNow connection itself uses the app's client secret via client-credentials — configure that in ServiceNow, not here.) EXPECTED RESULTS Green PASS lines for: Graph connect, each expected scope, and a managed-device read count. Any red FAIL tells you exactly what to fix (missing scope, no consent, no devices). .NOTES Intune Solutions — read-only. Cloud-aware (Global / USGov / USGovDoD). > [CmdletBinding()] param([ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

SNOWINT-Remediation.ps1

Purpose: The most common cause of a dead ServiceNow feed is the Entra app's client secret expiring or a required Graph scope not being consented. This script DIAGNOSES that (reports secrets near or past expiry and which expected app-role scopes are missing) and, only with -Apply, will create a fresh client secret so you can update the ServiceNow connection. It NEVER deletes an existing secret and NEVER grants consent silently. HOW TO RUN Diagnose (default, -WhatIf-safe, no changes): powershell -ExecutionPolicy Bypass -File .\SNOWINT-Remediation.ps1 -AppId <client-id> [-Cloud Global|USGov|USGovDoD] Create a new secret (explicit opt-in): .\SNOWINT-Remediation.ps1 -AppId <client-id> -Apply -SecretMonths 24 Requires Microsoft.Graph SDK and Application.ReadWrite.All (for -Apply) — an admin action. EXPECTED RESULTS A report of secret expiry + missing scopes. With -Apply, a NEW client secret value printed once (store it, then paste into the ServiceNow connection). Consent for any missing scope is surfaced as a manual step (by design) — grant it in the Entra portal. .NOTES Intune Solutions. Read-only by default; -Apply performs ONE additive change (new secret) with ShouldProcess. Cloud-aware. Grant consent is intentionally left to a human in the portal. > [CmdletBinding(SupportsShouldProcess=$true)] param([Parameter(Mandatory=$true)][string]$AppId, [switch]$Apply, [int]$SecretMonths = 24, [int]$WarnDays = 30, [ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

How to run: Diagnose (default, -WhatIf-safe, no changes): powershell -ExecutionPolicy Bypass -File .\SNOWINT-Remediation.ps1 -AppId <client-id> [-Cloud Global|USGov|USGovDoD] Create a new secret (explicit opt-in): .\SNOWINT-Remediation.ps1 -AppId <client-id> -Apply -SecretMonths 24 Requires Microsoft.Graph SDK and Application.ReadWrite.All (for -Apply) — an admin action. EXPECTED RESULTS A report of secret expiry + missing scopes. With -Apply, a NEW client secret value printed once (store it, then paste into the ServiceNow connection). Consent for any missing scope is surfaced as a manual step (by design) — grant it in the Entra portal. .NOTES Intune Solutions. Read-only by default; -Apply performs ONE additive change (new secret) with ShouldProcess. Cloud-aware. Grant consent is intentionally left to a human in the portal. > [CmdletBinding(SupportsShouldProcess=$true)] param([Parameter(Mandatory=$true)][string]$AppId, [switch]$Apply, [int]$SecretMonths = 24, [int]$WarnDays = 30, [ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

SNOWINT-Report-Coverage.ps1

Purpose: Summarizes the Intune managed-device estate the CMDB feed should reflect: counts by platform (Windows/macOS/iOS/Android), by ownership (corporate/personal), by compliance state, and how many devices have a stale last check-in. Use it to set expectations for CMDB coverage and to track the feed wave over wave during a Workspace ONE -> Intune transition. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SNOWINT-Report-Coverage.ps1 ` [-StaleDays 30] [-CsvPath C:\Temp\coverage.csv] [-Cloud Global|USGov|USGovDoD] Requires the Microsoft.Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS On-screen breakdowns (platform / ownership / compliance / stale) and an optional CSV. The totals should match the device count that lands in the ServiceNow CMDB. .NOTES Intune Solutions — read-only. Cloud-aware (Global / USGov / USGovDoD). > [CmdletBinding()] param([int]$StaleDays = 30, [string]$CsvPath, [ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\SNOWINT-Report-Coverage.ps1 ` [-StaleDays 30] [-CsvPath C:\Temp\coverage.csv] [-Cloud Global|USGov|USGovDoD] Requires the Microsoft.Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS On-screen breakdowns (platform / ownership / compliance / stale) and an optional CSV. The totals should match the device count

that lands in the ServiceNow CMDB. .NOTES Intune Solutions — read-only. Cloud-aware (Global / USGov / USGovDoD). > [CmdletBinding()] param([int]$StaleDays = 30, [string]$CsvPath, [ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

SNOWINT-Validate-Sync.ps1

Purpose: Proves the ServiceNow feed is correct and de-duplicated. Reads live Intune managed devices and a CSV the admin exports from the ServiceNow CMDB, then reports: matched (in both), missing in CMDB (in Intune, not yet imported), stale in CMDB (in CMDB, not in Intune), and DUPLICATE serials in the CMDB (an IRE / identifier problem to fix). HOW TO RUN 1) Export the CMDB device list to CSV with at least a serial-number column (e.g. a report on cmdb_ci_computer + the handheld class; column name passed via -CmdbSerialColumn). 2) powershell -ExecutionPolicy Bypass -File .\SNOWINT-Validate-Sync.ps1 ` -CmdbCsv C:\Temp\cmdb-devices.csv [-CmdbSerialColumn serial_number] [-Cloud Global|USGov|USGovDoD] Requires the Microsoft.Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Counts + a breakdown for matched / missing-in-CMDB / stale-in-CMDB / duplicate serials, with optional detail CSVs. Zero duplicates and near-zero missing = a healthy, reconciled feed. .NOTES Intune Solutions — read-only (reads Graph + a local CSV; writes only report CSVs). Cloud-aware. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$CmdbCsv, [string]$CmdbSerialColumn = 'serial_number', [string]$OutDir = ".", [ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')

How to run: 1) Export the CMDB device list to CSV with at least a serial-number column (e.g. a report on cmdb_ci_computer + the handheld class; column name passed via -CmdbSerialColumn). 2) powershell -ExecutionPolicy Bypass -File .\SNOWINT-Validate-Sync.ps1 ` -CmdbCsv C:\Temp\cmdb-devices.csv [-CmdbSerialColumn serial_number] [-Cloud Global|USGov|USGovDoD] Requires the Microsoft.Graph SDK and DeviceManagementManagedDevices.Read.All. EXPECTED RESULTS Counts + a breakdown for matched / missing-in-CMDB / stale-in-CMDB / duplicate serials, with optional detail CSVs. Zero duplicates and near-zero missing = a healthy, reconciled feed. .NOTES Intune Solutions — read-only (reads Graph + a local CSV; writes only report CSVs). Cloud-aware. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$CmdbCsv, [string]$CmdbSerialColumn = 'serial_number', [string]$OutDir = ".", [ValidateSet('Global','USGov','USGovDoD')][string]$Cloud = 'Global')